

Lab 4 - Vibe Coding

Complete Chat Reference with Prompts and Implementation Evidence

Name	Dhruv Swatantramath
SRN	PES1UG24CS1554
Project	Python/Pygame Centipede Game

Purpose

This document records the prompts used during the incremental Vibe Coding process, together with the implementation decisions and final verification. It is intended as a reference/proof document for the Software Engineering Lab 4 submission.

Prompt 1 - Initial Coding Assistant Instructions

I am completing Lab 4 - Vibe Coding for my Software Engineering lab.

My details:

Name: Dhruv Swatantramath

SRN: PES1UG24CS1554

The project is a Python/Pygame Centipede game.

I need to produce a clean chat history for submission, so please act as my coding assistant throughout this task.

I will provide the starter game.py code below.

Your job is to help me implement the four required changes in a clear, incremental way:

1. Fix the mushroom durability bug so a mushroom with 4 HP actually takes 4 shots to disappear.
2. Implement `mushroom_color(hp)` so mushroom color visually indicates its remaining HP.
3. Implement `on_segment_hit(segment, score)` to create a visible hit/spark effect when a Centipede segment is destroyed, without breaking chain splitting or scoring.
4. Implement `wave_speed_bonus(wave)` so later waves progressively move faster.

IMPORTANT:

- Preserve existing gameplay unless a change is necessary.
- Make minimal, understandable changes.
- Explain the bug/reasoning before giving code.
- After each change, explain exactly what was changed.
- Do not introduce unnecessary libraries or major architectural changes.
- At the end, provide the complete final game.py.
- Also give me a short testing checklist for demonstrating each feature in an AFTER video.

Here is my starter code: [starter game.py supplied in the chat].

Prompt 2 - Change 2

Change 1 is understood and the mushroom durability fix is complete.

Now move to Change 2: implement `mushroom_color(hp)`.

Requirements:

- The mushroom should visually indicate its remaining HP.
- Use different colors for different HP values.
- Keep the existing drawing logic intact.
- Do not change mushroom collision, scoring, or durability behavior.
- Do not introduce external assets or libraries.
- Make the smallest reasonable change.

First explain the HP-to-color mapping you will use.

Then provide only the updated `mushroom_color(hp)` function and explain how the existing `draw()` method will use it.

Prompt 3 - Change 3

Change 2 is complete.

Now implement Change 3: `on_segment_hit(segment, score)`.

Requirements:

- When a Centipede segment is successfully shot, show a brief visible spark/particle effect at the segment's position.
- The effect should be temporary rather than permanent.
- Do not break chain splitting.
- Do not change the existing scoring rules.
- Do not use external image or sound assets.
- Keep the implementation simple and compatible with the existing Pygame structure.
- Make the smallest reasonable changes.

First explain how you will implement the effect.
Then provide the necessary code changes.

Prompt 4 - Change 4

Change 3 is complete.

Now implement Change 4: `wave_speed_bonus(wave)`.

Requirements:

- Later waves should make the Centipede progressively faster.
- Keep the existing `effective_tick` calculation working.
- Use a simple predictable progression.
- Do not change unrelated movement logic.

Use this progression:

Wave 1 = 1.00x

Wave 2 = 1.15x

Wave 3 = 1.30x

Wave 4 = 1.45x

Wave 5 = 1.60x

Explain how the multiplier affects `effective_tick` and why later waves move faster.
Then provide the updated function.

Prompt 5 - Complete Final game.py

All four changes have now been implemented incrementally.

Now integrate the changes into the complete final `game.py`.

Please:

1. List the four implemented changes.
2. Provide the complete final `game.py`.
3. Preserve all existing gameplay functionality.
4. Do not introduce unrelated changes.
5. Make sure the code is internally consistent and runnable.

Prompt 6 - Final Software Engineering Review

Review the complete final `game.py` as a Software Engineering lab submission.

Check specifically:

- Mushroom requires exactly 4 hits before disappearing.
- Mushroom color reflects remaining HP.
- Centipede segment hit produces a temporary visible effect.
- Chain splitting still works.
- Existing scoring remains correct.
- Later waves become progressively faster.
- Player movement, shooting, lives, reset, and game-over behavior remain intact.
- No obvious runtime or logic errors were introduced.

If anything is wrong, explain it and provide the smallest necessary fix.
Otherwise, explicitly state that the code is ready for testing.

Implementation Evidence / Final Decisions

1. Mushroom durability: `hit_mushroom()` decrements HP and deletes the mushroom only when HP reaches 0.
2. Mushroom colors: 4 HP = blue, 3 HP = green, 2 HP = yellow, 1 HP = red.
3. Segment hit effect: eight short-lived particles are created at the destroyed segment position and removed after their lifetime expires.
4. Wave speed: `wave_speed_bonus(wave)` returns $1.0 + (\text{wave} - 1) * 0.15$, producing 1.00x, 1.15x, 1.30x, 1.45x and 1.60x for waves 1–5.
5. Final review cleanup: `hit_particles.clear()` should be placed at the start of `Game.reset()` so reset also clears any active spark particles.

AFTER Video Testing Checklist

- Shoot one mushroom four times and show that it disappears only on the fourth shot.
- Demonstrate the mushroom color progression as HP decreases.
- Shoot a Centipede segment and capture the temporary spark effect.
- Shoot a middle segment and demonstrate chain splitting.
- Demonstrate that existing scoring remains correct.
- Reach later waves and demonstrate progressively faster Centipede movement.
- Demonstrate player movement, shooting, lives/respawn, reset, and game-over behavior.

Final status: reviewed and ready for testing after the small reset cleanup.